

Lab01

1 Task 1: Height of LBJ

Write a program to convert one's height from US Customary Units to SI Units
Test with LBJ's data or any other one you like.

2 Task 2: $A+B$

2.1 Most used type

Type	Format	Declaration
char	%c	char x=' a' ;
int	%d or %i	int x=1;
(base 8)	%o	int x=03;
(base 16)	%x	int x=0x123;
unsigned int	%u	unsigned int x=2;
float	%f	float x=1.25
double	%lf	double x=1.25 (scanf)

2.2 Reading and Writing Integers

The number associated with the format in which you read and write

```
int main(int argc, char** argv)
{
    unsigned int u;
    printf("Input a number\n");
    scanf("%u", &u); //! Try to input 2147483646
    printf("In base 10: %u\n", u);
    printf("In base 10: %d\n", u);
    printf("Input a number\n");
    scanf("%o", &u);
    printf("In base 8: %o\n", u);
    printf("In base 10: %d\n", u);
    printf("Input a number\n");
    scanf("%x", &u);
    printf("In base 16: %x\n", u);
    printf("In base 10: %d\n", u);

    /** Normally exiting the program, return 0 **/
    return 0;
}
```

then input 10
record the results.

Try to input 2147483648

2.3 Formatted Input/Output

`scanf("%c %f", &type, &temp)`

`scanf` is controlled by the conversion specification in the format string (in "...") starting from left to right.

When called, it tries to locate an item of the appropriate type (eg. %d, %f) in the input data, **skipping white-space characters**

- the space,
- horizontal and vertical tab,
- form-feed,
- and new-line character

Pay very attention to "%c"

White-space characters:

- one white-space character in the format string will match any number of white-space character in the input.

Other characters:

- when it encounters a non-white-space character in a format string, `scanf` compares it with the next input character.
- If the two characters match, `scanf` discards the input character and continues processing the format string.
- Else, `scanf` puts the offending character back into the input, then aborts without further processing.
- **test:**

`%d/%d` will match `_5/_96`, but not `_5/_96`

`%d_/%d` will match `_5/_96`

Which `_` represents the space

当在表达式里出现一个非空白字符，`scanf` 会去和下一个输入字符比较：

如果相同，往下继续；

如果不同，把读取字符放回输入流，放弃继续处理。

```
int main(int argc, char** argv)
{
    //!
    int a, b;
    scanf("%d/%d",&a, &b);
    //! try to input "_5/_96" and "_5/_96"
    //! record the results
    printf("a and b are: %d, %d", a, b);

    scanf("%d /%d",&a, &b);
    //! try to input "_5/_96" and "_5/_96"
    //! record the results
    printf("a and b are: %d, %d", a, b);

    /** Normally exiting the program, return 0 **/
    return 0;
}
```

2.4 设计一段程序，从键盘读入 A 、 B 两个数。计算 $A+B$ 的值并输出

描述：

A 、 B 均为整数

提示：

使用 `scanf("%d %d", &A, &B);` 可以从键盘读出用空格分割的两个整数

使用 `printf("%d", A+B);` 可以输出计算的结果

尝试普通案例之后，试一下输入 $a = 2147483646$ ， $b = 1, 2, 3$ 的结果

演示：

```
int main(int argc, char** argv)
{
    /** Define two integer variables **/
    int a, b;

    /** Use scanf to get values from stdin
     *
     * - scanf() read keyboard input
     * - "%d" is the pattern for integer input
     * - &a represent the address of variable a, scanf() modifies a via this address
     **/
```

```
printf("please input two int numbers:\n");
scanf("%d %d", &a, &b);

/** Use printf to put values to stdout
 *
 * - printf() prints
 * - "%d" is the pattern for integer output
 * - a + b is the value of answer, printf use it to format output
 */
// Try a = 2147483646, b = 1, 2, 3. See what happened.
printf("a + b = %d\n", a + b);

/** Normally exiting the program, return 0 */
return 0;
}
```
